

EE272 Homework 1

Accelerator Design Space Exploration Report

Student Name 1: Berk Gokmen
Student Name 2: Edison Altamirano

Please answer the questions below in the space provided and include the code/reports indicated in your submission.

1. Basic Approach

- 1.1 Start with your basic approach. How did your group decide to attack the problem? How did you find hardware that works for the overall problem, when the tool only optimizes a single layer at a time? 10 points

We started by defining a practical strategy to handle the fact that Interstellar only optimizes one layer at a time. First, we generated the ResNet-18 layer JSONs described in the paper and decided to begin the architecture search using the most computationally expensive layer, which we determined was the conv2 layer (explained in more detail in question 1.2). The underlying assumption here was that the optimal architectures for that layer would be close to the optimal for all layers. To find the best architecture for that layer, we performed a parameter sweep over different architectures within a design subspace (explained in more detail in question 1.3) by varying `num_pe`, `rf_size`, and `buffer_size` with the appropriate memory access costs from Tables 1 and 2. For each architecture, we ran the optimizer and stored the results in a text file with the format `num_pe, rf_size, buffer_size, energy_pJ, runtime_cycles, elapsed_s`, allowing us to rank architectures and select the top candidates based on minimum energy and minimum energy cycles. After that, we evaluated these best candidates across the top four most computationally expensive layers in ResNet-18: layer3 (conv2), layer7 (conv3), layer11 (conv4), and layer15 (conv5). Since these layers were reused three times in the network, they provided a great approximation for the whole network. Finally, the best candidates from these results were evaluated across the entire network. We summed energy across these representative layers for each of the top architectures, compared the resulting total energy, and selected the architecture that achieved the best overall FoM for the full DNN. Overall, since parameter search was very time-consuming, we created an evaluation funnel where many architectures were evaluated for one layer, then a subset of those were evaluated for four layers, and the final few were evaluated over all layers. We believe this provided a good trade-off between computation time and reaching the true optimum.

1.2 How did you decide on which layers to look at first, and why did you choose these layers? List the number of MACs for each layer.

10 points

We selected the most computationally expensive layers first because these stages repeat and dominate overall cost. These were conv2, conv3, conv4, and conv5. The calculated number of MACs for each layer are shown in Table 1 below. Although layer 1 is the most computationally expensive layer on its own (~1.9G MACs), the repeating layers in conv2 through conv5 make up for the majority of the network's total computation (For example, conv2.x alone is about 7.4G MACs). Therefore, using the table below, we decided to first look at the conv2 layer alone followed by the combination of the four critical layers conv2, conv3, conv4, and conv5. Only at the very end we decided to evaluate the architecture across all layers.

The formula used to calculate the MACs was $H_{output} \cdot W_{output} \cdot C_{output} \cdot C_{input} \cdot H_{filter} \cdot W_{filter} \cdot Batch$

Layer #	Layer Name	MACs
Layer 1	conv1	1,888,223,232
Layer 2	conv2_1_1	1,849,688,064
Layer 3	conv2_1_2	1,849,688,064
Layer 4	conv2_2_1	1,849,688,064
Layer 5	conv2_2_2	1,849,688,064
Layer 6	conv3_1_1	924,844,032
Layer 7	conv3_1_2	1,849,688,064
Layer 8	conv3_2_1	1,849,688,064
Layer 9	conv3_2_2	1,849,688,064
Layer 10	conv4_1_1	924,844,032
Layer 11	conv4_1_2	1,849,688,064
Layer 12	conv4_2_1	1,849,688,064
Layer 13	conv4_2_2	1,849,688,064
Layer 14	conv5_1_1	924,844,032
Layer 15	conv5_1_2	1,849,688,064
Layer 16	conv5_2_1	1,849,688,064
Layer 17	conv5_2_2	1,849,688,064

Table 1: Table listing the calculated number of MACs per layer.

1.3 What part of the design space did you explore, and why did you choose that subspace?

5 + 5 points

We explored the hardware design space defined by three architecture parameters: number of PEs, RF size, and buffer size, since these directly control compute parallelism, on-chip storage and reuse, and memory movement (which dominates energy). We restricted the sweep to discrete, power-of-two values for RF and buffer sizes because the HW cost model provides access-cost values only at those points and because these sizes represent realistic SRAM configurations. Additionally, we only considered RF sizes between 16 B and 512 B, as well as buffer sizes between 8 KB and 512 KB. This is because sizes under this range would not provide sufficient data reuse, and sizes above this range would be dominated by access cost. After restricting the memory configuration to these limitations, we explored all 3-level hardware architectures with square PE counts and with memory configurations that fall in the mentioned range, with $1 \text{ mm}^2 \leq \text{area} \leq 2 \text{ mm}^2$. The area upper bound of 2 mm^2 is enforced by the problem statement. We set the area lower bound to 1 mm^2 with the assumption that designs that consume very little area are not optimal, since they leave a lot of additional area that could be used for resources on the table. This constrained subspace allowed us to efficiently evaluate a large set of valid candidates while still capturing the main tradeoffs between compute throughput and memory energy.

In addition to 3-level memory exploration (RF, SRAM buffer, DRAM), we also considered extending the accelerator to a 4-layer memory architecture consisting of two levels of register files (RF0 and RF1), followed by a shared on-chip buffer and off-chip DRAM. The motivation is to increase near-compute reuse by introducing an intermediate RF layer (RF1) between the MAC-local RF0 and the global buffer. This enables a more granular tiling strategy in which larger tiles can be staged in RF1 while micro-tiles remain in RF0, thereby reducing expensive accesses to the shared buffer and DRAM. In our implementation, we parameterize candidate architectures by (num_pe, total_rf, buffer_size) and split total_rf into (RF0, RF1) using a set of predefined RF splits (16, 32), (32, 32), (16, 64), (32, 64), (64, 64). This split strategy was chosen to keep the exploration tractable and consistent with the tool's supported RF capacities (powers-of-two RF sizes) while preserving the interpretation of RF0 as a smaller, MAC-proximal storage and RF1 as a larger staging RF for reuse. Finally, to reduce runtime while still evaluating meaningful candidates, we tested this architecture using only the Top-5 (num_pe, rf_size, buffer_size) configurations identified in our previous sweep, and evaluated each candidate across multiple critical ResNet-18 layers to approximate network-level performance and energy trends.

Before exploring the 4-layer memory hierarchy, the best-performing architecture under the area constraint was (num_pe=441, rf_size=64, buffer_size=65536), achieving a total energy of 59.89 mJ. After enabling and evaluating the 4-layer memory architecture (RF0 + RF1 + Buffer + DRAM), the optimal architecture became (num_pe=441, rf_size=48, buffer_size=65536) with a significantly lower total energy of 35.01 mJ. This represents an energy reduction of approximately 41.6%, showing that introducing an additional RF level improves data reuse closer to compute and reduces expensive accesses to higher memory levels (buffer/DRAM). The results indicate that the memory hierarchy design and RF organization strongly impact system-level energy, even when PE parallelism remains similar.

A) Results before 4-layer memory (baseline / 3-level memory results)

num_pe	rf_size	buffer_size	area_mm2	total_energy_pJ	total_energy_mJ
441	64	65536	1.9949	5.98933e+10	59.8933
256	64	131072	1.4557	6.16504e+10	61.6504
289	64	131072	1.5893	6.17986e+10	61.7986
324	64	131072	1.731	6.18023e+10	61.8023
361	64	131072	1.8808	6.18061e+10	61.8061

B) Results after 4-layer memory exploration

num_pe	rf_size	buffer_size	area_mm2	total_energy_pJ	total_energy_mJ
441	48	65536	1.7691	3.50052e+10	35.0052
289	96	131072	1.8852	3.77551e+10	37.7551
289	80	131072	1.7373	4.48328e+10	44.8328
324	80	131072	1.8969	4.50305e+10	45.0305
289	48	131072	1.4413	7.39246e+10	73.9246

- 2.1 Given your strategy, how did you implement that? What tools did you use, whether it is modifying the source code, writing your own script, or using an excel sheet? You should describe how to replicate your approach. How did you fit in the area constraint? How did your plan work (be honest)?

We implemented everything in Python. The following scripts were created:

calc_macs.py:

This script takes in a directory that contains all the ResNet-18 layer JSONs and calculates the number of MACs for each layer using the formula $MACs = H_{output} \cdot W_{output} \cdot C_{output} \cdot C_{input} \cdot H_{filter} \cdot W_{filter} \cdot Batch$

parameter_search.py:

This file is responsible for iterating over a range of parameters defined by the user. It has 2 modes: one for 3-level memory exploration and another for 4-level exploration. This search file runs user defined number of threads in parallel. By obtaining a 16 core machine on FarmShare and launching 16 threads in parallel, we were able to get a 16x speedup by running many optimizer scripts in parallel. Each thread generates a temporary architecture JSON file and a temporary schedule JSON file. These temporary files are deleted after the run is complete.

3-level exploration:

The user provides minimum and maximum sizes for the number of PEs, size of RF, and size of the buffer. The user also provides a specific layer and a dataflow schedule JSON file. Finally, the user provides the minimum and maximum area that each architecture must fall under. After specifying these specific values, the script iterates through all possible combinations of number of PEs (must be a square number) and memory configurations (must be a power of 2). Then, if the design's area falls within the acceptable memory bounds, the script creates a temporary hardware architecture file and runs the optimizer. The script also updates the schedule partitioning size to be the square root of the number of PEs. After the run is finished, each thread appends the results to the results file.

4-level exploration:

The user provides the top 5 designs from the previous 3-level memory exploration in order to investigate whether using a 4-level configuration would be more beneficial. The script uses the same exact steps as for the 3-level exploration process; however, it only iterates through the 5 designs provided from the user. Then, for these 5 designs, the script tries the following splits (16, 32), (32, 32), (16, 64), (32, 64), (64, 64) for level 0 and level 1 memory. In this mode, the script writes a 4-level memory architecture by setting mem_levels to 4 and writing capacity and access_cost for RF0, RF1, buffer, and DRAM. After the run is finished, each thread appends the results to the results file.

eval_arches_multi_layer.py:

This script takes in a list of layers provided by the user, and evaluates a list of architectures provided by the user across the provided layers. Then, the results for each architecture is added to the results file. This script was used when evaluating designs across critical layers, as well as evaluating designs across the entire network.

Initially, our plan of exploring many parameters for the first layer didn't work very well because runs would take over 24 hours. However, after facing this challenge, we realized we could parallelize the computation to 16 cores and get a 16x speedup. After this change, our plan worked very well because all the runs would take at most 3 hours to complete, and we were able to experiment with many different configurations and architectures.

3. Results

- 3.1 Present your results. What are the optimal dataflow, size of PE array, number of levels in memory hierarchy, and memory capacity at each level and how do you make these design choices? Here it is better if you present what your results mean. That is, rather than just presenting a table of numbers, try to talk about important trends you see from the results – are many designs very similar to each other, or is there a strong optimal point; what is the most critical aspect to optimize; does the area constraint matter; etc. 25 points

We begin our analysis by presenting our results from exploring architectures targeted towards the conv2 layer for the 3-level memory configuration. Table 2 below lists the top 5 optimal architectures for conv2. One important trend immediately seen by the table is that the most optimal designs use up the entire area, the minimum area being 1.85 mm² and the maximum area being 1.99 mm². This result suggests that a higher allowed area would yield designs that achieve higher FoM values and also verifies our assumption that designs that underutilize resources wouldn't be optimal.

Number of PEs	RF Size (B)	Buffer Size (B)	Area (mm ²)	Energy (mJ)	Runtime (Cycles)	J·cycles
576	32	65,536	1.95	4.22	3,211,264	1.35e4
441	64	65,536	1.99	3.59	4,194,304	1.51e4
625	32	32,768	1.99	5.00	3,082,814	1.54e4
576	32	32,768	1.85	5.00	3,211,264	1.60e4
676	16	65,536	1.91	5.58	2,964,244	1.66e4

Table 2. Table listing the top 5 optimal configurations for the 3-level memory configuration for conv2

Looking at the top five designs in more detail, they all have the same buffer size (64 KB or 32 KB) and similar RF sizes (64 to 32 bytes). To further investigate this similarity, we created Figure 1, Figure 2, and Figure 3, shown below, to examine how the FoM changes for various PE, RF, and buffer sizes. Looking at Figure 1 in detail, it is clear that designs between 400 and 750 PEs have very similar FoM values, which suggests that most designs within this range are optimal and that there is no single strong optimum that stands out. Similarly, for Figure 2, designs with RF sizes between 16 and 64 bytes and, based on Figure 3, buffer sizes below 128 KB fall within a similar optimal region. These results strongly suggest that there is no single design that performs drastically better given a specific architectural parameter.

However, looking at all the figures below, it is obvious that the number of PEs is the most critical parameter to optimize. Looking at Figures 2 and 3, there is no drastic difference between the optimal FoM when architectures are restricted to a specific memory configuration; however, restricting the number of PEs does create a significant difference in performance and energy efficiency.

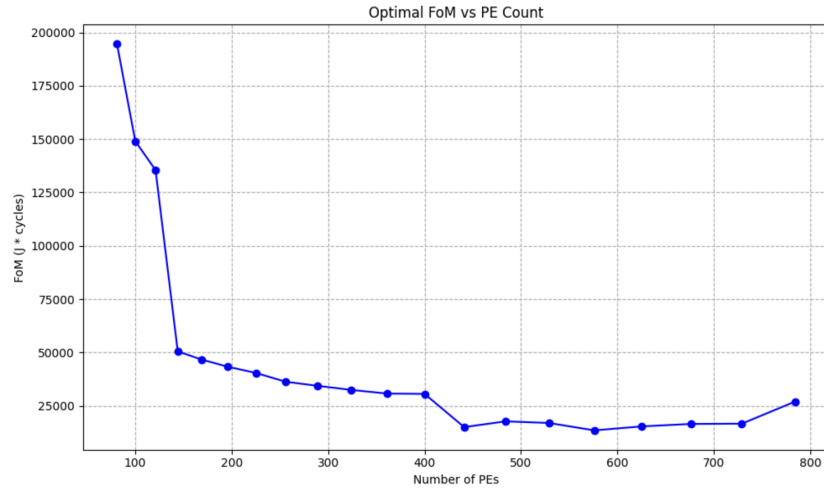


Figure 1. Graph showing how the optimal FoM (Joules * cycles) varies with the number of PEs. Optimal FoM here is defined as the architecture with 3-level memory configuration that achieves the highest FoM for the given number of PEs.

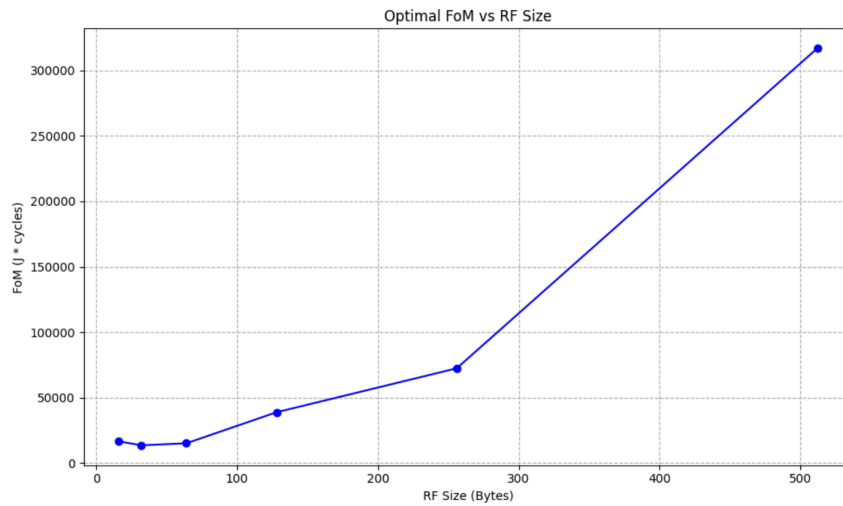


Figure 2. Graph showing how the optimal FoM (Joules * cycles) varies with the RF size. Optimal FoM here is defined as the architecture with 3-level memory configuration that achieves the highest FoM for the given RF size.

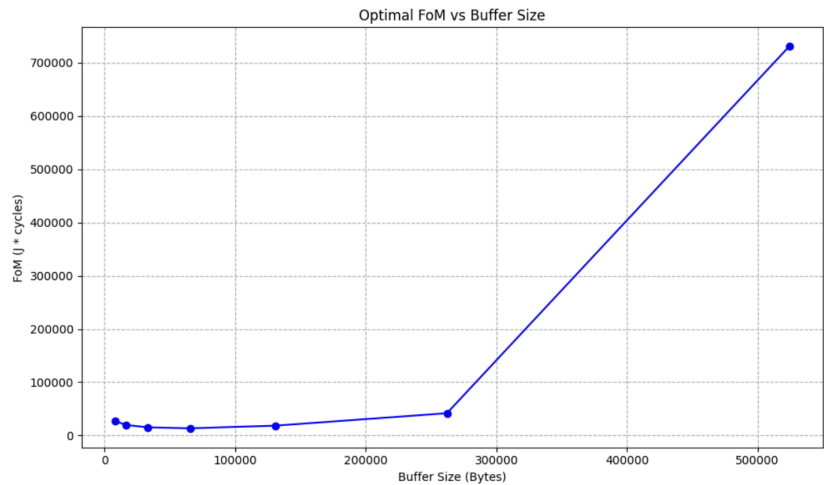


Figure 3. Graph showing how the optimal FoM (Joules * cycles) varies with the buffer size. Optimal FoM here is defined as the architecture with 3-level memory configuration that achieves the highest FoM for the given buffer size.

The strongest “optimal point” appears when we combine high PE count (441) with a memory hierarchy that enables near-compute reuse. This is most visible in the 4-layer results: keeping the same top-level PE count (441) but changing the RF organization from a single RF to two RF levels produced a large energy drop (from ~59.9 mJ to ~35.0 mJ). This suggests that the accelerator is memory-energy dominated, and architectural changes that reduce buffer/DRAM traffic have an outsized impact. Notably, the best 4-layer design achieves much lower energy while still staying comfortably within the area limit (~1.769 mm²), showing that architectural organization (adding RF level) can outperform brute-force scaling under tight area budgets. We analyzed the energy percentage per critical layer for the best architecture before and after exploring 4-layer memory to understand what drives total network energy. Before 4-layer memory, energy is highly concentrated, with a few layers dominating the total (the largest layer contributing ~25%). After enabling the 4-layer hierarchy (RF0 + RF1 + Buffer + DRAM), the energy distribution across the dominant layers becomes more balanced (~21–23% each), suggesting that the additional RF level improves near-compute reuse and reduces expensive buffer/DRAM accesses in the most energy-intensive layers. This supports both our critical-layer evaluation approach and the benefit of the 4-layer memory configuration.

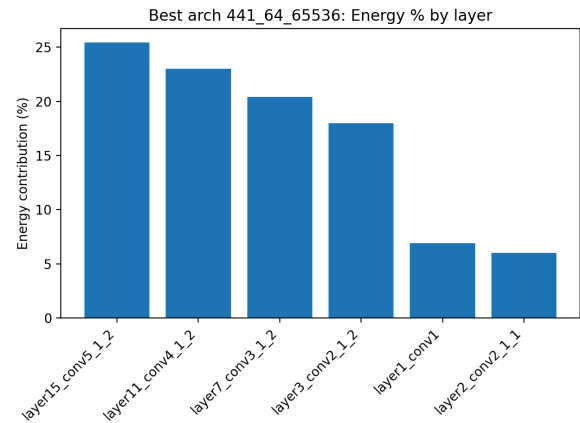


Figure 4. Layer analysis before 4-layer memory (baseline / 3-level memory results)

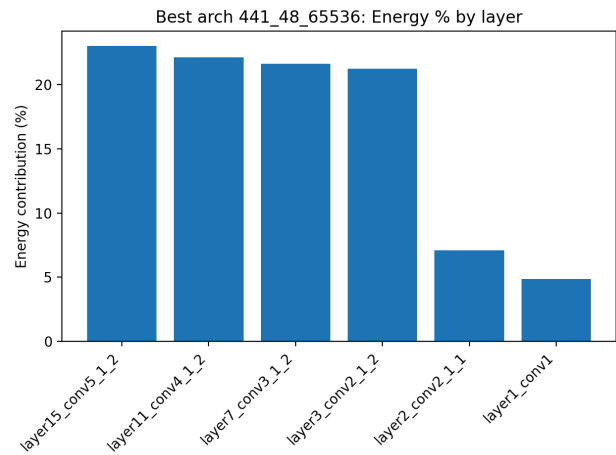


Figure 5. Layer analysis after 4-layer memory exploration

4. Summary

- 4.1 Finally, summarize the following parameters of your optimal 2mm² design in the table below. 25 points
Energy and cycles in the table must correspond to that of the entire ResNet-18 network (all CONV layers) on your accelerator.

Table 3: Optimal accelerator parameters. If you have more levels in the memory hierarchy, specify each of their sizes and clearly state if the size is per PE or the total for all PEs.

Most Optimal 3-Layer Configuration:

Number of PEs	576
Register File Size Level 0 (Bytes)	32
Global Buffer Size (Kbytes)	64KB
Energy of All Layers (mJ)	74.789269734400009 mJ
Cycles of All Layers (million)	51.21775 million cycles
Area (mm ²)	1.9515392 mm ²
Total Energy * Total Cycles	3.83e6

Most Optimal 4-Layer Configuration:

Number of PEs	441
Register File Size Level 0 (Bytes)	16
Register File Size Level 1 (Bytes)	32
Global Buffer Size (Kbytes)	64KB
Energy of All Layers (mJ)	35.00516907328 mJ
Cycles of All Layers (million)	58.807638 million cycles
Area (mm ²)	1.7691 mm ²
Total Energy * Total Cycles	2.06e6